

CMPT 383: Vitamin #0

Anders Miltner
miltner@cs.sfu.ca

Due Sep 14

Introduction

This Vitamin is here just to help you set up and install Haskell, set up and install your IDE, and learn some commands for interacting with Haskell.

There is no submission or grading for this Vitamin. You should attend office hours if you have difficulties with setup.

1 Installing GHCUp

Download GHCUp using the installation instructions from <https://www.haskell.org/ghcup/install/>.

You should install GHCUp with all the default options.

GHCUp will not install its dependencies. GHCUp will give information on what dependencies you must download. Please follow GHCUp's instructions and install those dependencies.

2 Installing VS Code

Throughout the class, we'll use Visual Studio Code as our default text editor. I will be demoing on the VS Code editor throughout the class. If you want to use a different text editor, that is fine, but we won't be able to provide much "Tech Support" for installation and use.

Download VS Code from <https://code.visualstudio.com/download>.

3 Setting up a Haskell Environment Within VS Code

1. Open VS Code.
2. Click the "Extensions" tab
3. Search "Haskell"
4. Click "Install" on the "Haskell" extension, by developer "Haskell"
5. Click File > Open Folder, and navigate to the unzipped v0 directory that this pdf was inside
6. Navigate to the "src" directory, and open "Lib.hs"
7. The extension will request permission to download various files, which should be granted
8. Wait for the extension to install all files and set up the project. This may take a few hours for the first time. The setup is done when there are small "Refresh" clickable items underneath `helloWorldString`.

Note, some may hit issues with Apple Silicon Macs. The stackage snapshot I included uses GHC 9.0.2, which requires LLVM version 9-12 (inclusive) to build on AArch64. If you try to install LLVM or already have it installed, they likely have version 13+.

The solution is to make sure LLVM version 12 is installed and then make sure the GHCup builds GHC 9.0.2 using that version. If you use homebrew, this can be done as follows:

1. `stack purge` (if you already ran into a build error)
2. `brew install llvm@12`
3. `OPT=/opt/homebrew/opt/llvm@12/bin/opt`
`LLC=/opt/homebrew/opt/llvm@12/bin/llc`
`ghcup install ghc 9.0.2 --force`
4. `stack build`

4 Running Your First Haskell Program

Like many languages, executing a Haskell program runs the code in the “main” function. We can compile and run a Haskell program in our IDE. Open up a new terminal in VS Code (Terminal > New Terminal).

The terminal should print out `v0 $`. Type `stack build` into the terminal and press enter. You should see a few lines of build information. To run the newly compiled program, type `stack run v0-exe`, which should output “Hello, World!”. Note, for some systems, a default “stack” will not have been installed. If that’s the case, try `stack-2.7.5 build` and `stack-2.7.5 run v0-exe`.

5 Running Haskell Test Suites

The Haskell build system we are using has built-in support for running test suites. In the open terminal, type `stack test`. This should output “(Empty) Test suite ran!”.

6 Getting Evaluation and Type Information via Haskell Language Server

The Haskell Language Server (HLS) that underlies the VS Code Haskell extension is quite powerful. The way the HLS permits human interaction is through comments. In this section, we will show how to extract evaluation and type information from the HLS using comments.

After the main function are 4 other comment snippets that interact with the HLS. The first one should look like `-- >>> helloWorldString`. The `--` at the start initiates a single-line comment, similar to `//` in Java and C. The `>>>` that follows initiates an interaction with the HLS. What type of information you get back depends on what follows.

In the first comment, we simply write some Haskell code after `>>>`. This signals to the HLS that you wish to evaluate that code, then print it out. In this example, it appends `!!` to `helloWorldString`. Click “Refresh” to evaluate that code snippet, and see the result.

The next comment begins with `-- >>>`, but then has `:t` before the Haskell code. This means that we wish to get the type of the code that follows. Click “Refresh” type-check that code snippet, and see the result.

The next comment is a different type of comment. It is a multi-line comment, similar to `/*` and `*/` in Java and C. Within multi-line comments, you can also interact with the HLS by adding `>>>`.

In the last line, we do another type-checking. But this doesn’t check a single variable, but a full expression. Just as you can evaluate expressions with the HLS, you can also type-check them! Try adding your own!